
RAG and Agents

To solve a task, a model needs both the instructions on how to do it, and the necessary information to do so. Just like how a human is more likely to give a wrong answer when lacking information, AI models are more likely to make mistakes and hallucinate when they are missing context. For a given application, the model's instructions are common to all queries, whereas context is specific to each query. The last chapter discussed how to write good instructions to the model. This chapter focuses on how to construct the relevant context for each query.

Two dominating patterns for context construction are RAG, or retrieval-augmented generation, and agents. The RAG pattern allows the model to retrieve relevant information from external data sources. The agentic pattern allows the model to use tools such as web search and news APIs to gather information.

While the RAG pattern is chiefly used for constructing context, the agentic pattern can do much more than that. External tools can help models address their shortcomings and expand their capabilities. Most importantly, they give models the ability to directly interact with the world, enabling them to automate many aspects of our lives.

Both RAG and agentic patterns are exciting because of the capabilities they bring to already powerful models. In a short amount of time, they've managed to capture the collective imagination, leading to incredible demos and products that convince many people that they are the future. This chapter will go into detail about each of these patterns, how they work, and what makes them so promising.

RAG

RAG is a technique that enhances a model's generation by retrieving the relevant information from external memory sources. An external memory source can be an internal database, a user's previous chat sessions, or the internet.

The *retrieve-then-generate* pattern was first introduced in “Reading Wikipedia to Answer Open-Domain Questions” (Chen et al., 2017). In this work, the system first retrieves five Wikipedia pages most relevant to a question, then a model¹ uses, or reads, the information from these pages to generate an answer, as visualized in Figure 6-1.

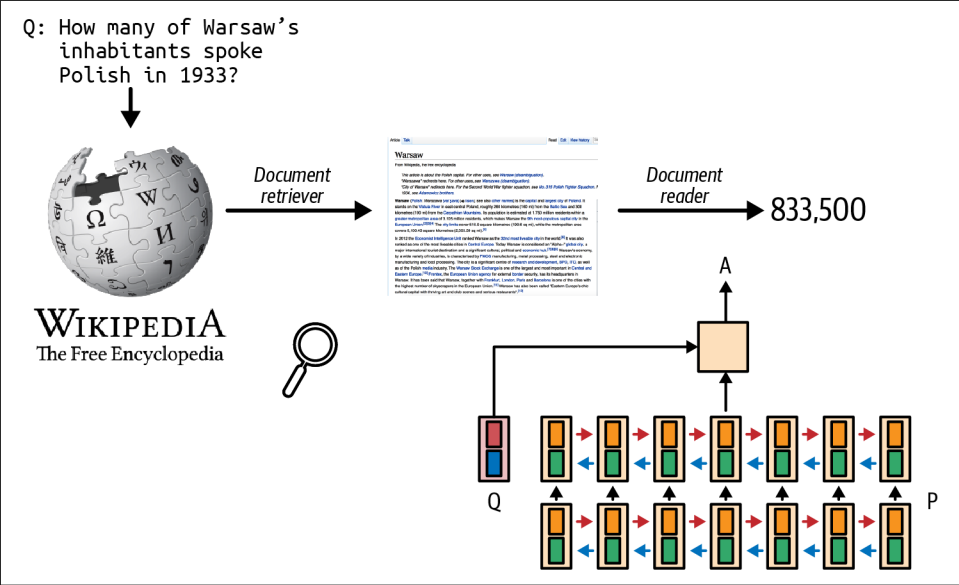


Figure 6-1. The retrieve-then-generate pattern. The model was referred to as the document reader.

The term retrieval-augmented generation was coined in “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (Lewis et al., 2020). The paper proposed RAG as a solution for knowledge-intensive tasks where all the available knowledge can’t be input into the model directly. With RAG, only the information most relevant to the query, as determined by the retriever, is retrieved and input into the model. Lewis et al. found that having access to relevant information can help the model generate more detailed responses while reducing hallucinations.²

1 The model used was a type of **recurrent neural network** known as **LSTM** (Long Short-Term Memory). LSTM was the dominant architecture of deep learning for natural language processing (NLP) before the transformer architecture took over in 2018.

2 Around the same time, another paper, also from Facebook, “How Context Affects Language Models’ Factual Predictions” (Petroni et al., *arXiv*, May 2020), showed that augmenting a pre-trained language model with a retrieval system can dramatically improve the model’s performance on factual questions.

For example, given the query “Can Acme’s fancy-printer-A300 print 100pps?”, the model will be able to respond better if it’s given the specifications of fancy-printer-A300.³

You can think of RAG as a technique to construct context specific to each query, instead of using the same context for all queries. This helps with managing user data, as it allows you to include data specific to a user only in queries related to this user.

Context construction for foundation models is equivalent to feature engineering for classical ML models. They serve the same purpose: giving the model the necessary information to process an input.

In the early days of foundation models, RAG emerged as one of the most common patterns. Its main purpose was to overcome the models’ context limitations. Many people think that a sufficiently long context will be the end of RAG. I don’t think so. First, no matter how long a model’s context length is, there will be applications that require context longer than that. After all, the amount of available data only grows over time. People generate and add new data but rarely delete data. Context length is expanding quickly, but not fast enough for the data needs of arbitrary applications.⁴

Second, a model that can process long context doesn’t necessarily use that context well, as discussed in “[Context Length and Context Efficiency](#)” on page 218. The longer the context, the more likely the model is to focus on the wrong part of the context. Every extra context token incurs extra cost and has the potential to add extra latency. RAG allows a model to use only the most relevant information for each query, reducing the number of input tokens while potentially increasing the model’s performance.

Efforts to expand context length are happening in parallel with efforts to make models use context more effectively. I wouldn’t be surprised if a model provider incorporates a retrieval-like or attention-like mechanism to help a model pick out the most salient parts of a context to use.

³ Thanks to Chetan Tekur for the example.

⁴ Parkinson’s Law is usually expressed as “Work expands so as to fill the time available for its completion.” I have a similar theory that an application’s context expands to fill the context limit supported by the model it uses.



Anthropic suggested that for Claude models, if “your knowledge base is smaller than 200,000 tokens (about 500 pages of material), you can just include the entire knowledge base in the prompt that you give the model, with no need for RAG or similar methods” (Anthropic, 2024). It’d be amazing if other model developers provide similar guidance for RAG versus long context for their models.

RAG Architecture

A RAG system has two components: a retriever that retrieves information from external memory sources and a generator that generates a response based on the retrieved information. Figure 6-2 shows a high-level architecture of a RAG system.

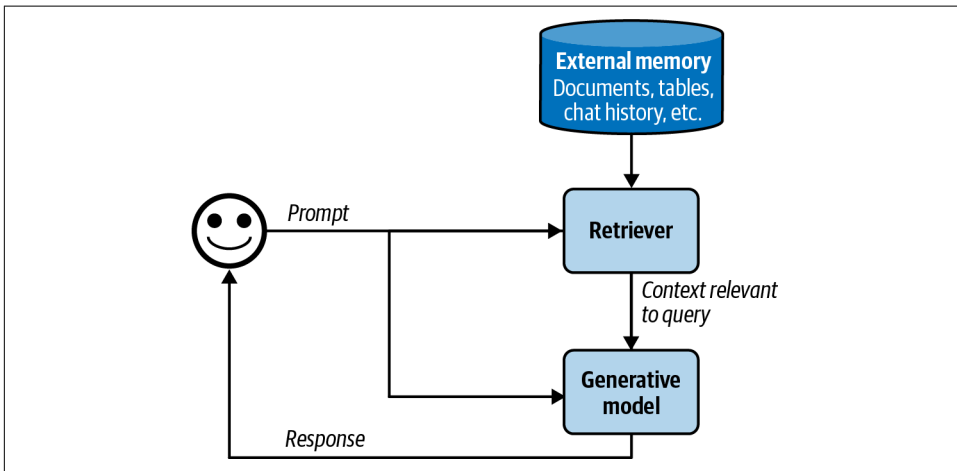


Figure 6-2. A basic RAG architecture.

In the original RAG paper, Lewis et al. trained the retriever and the generative model together. In today’s RAG systems, these two components are often trained separately, and many teams build their RAG systems using off-the-shelf retrievers and models. However, finetuning the whole RAG system end-to-end can improve its performance significantly.

The success of a RAG system depends on the quality of its retriever. A retriever has two main functions: indexing and querying. Indexing involves processing data so that it can be quickly retrieved later. Sending a query to retrieve data relevant to it is called querying. How to index data depends on how you want to retrieve it later on.

Now that we’ve covered the primary components, let’s consider an example of how a RAG system works. For simplicity, let’s assume that the external memory is a database of documents, such as a company’s memos, contracts, and meeting notes. A

document can be 10 tokens or 1 million tokens. Naively retrieving whole documents can cause your context to be arbitrarily long. To avoid this, you can split each document into more manageable chunks. Chunking strategies will be discussed later in this chapter. For now, let's assume that all documents have been split into workable chunks. For each query, our goal is to retrieve the data chunks most relevant to this query. Minor post-processing is often needed to join the retrieved data chunks with the user prompt to generate the final prompt. This final prompt is then fed into the generative model.



In this chapter, I use the term “document” to refer to both “document” and “chunk”, because technically, a chunk of a document is also a document. I do this to keep this book’s terminologies consistent with classical NLP and information retrieval (IR) terminologies.

Retrieval Algorithms

Retrieval isn’t unique to RAG. Information retrieval is a century-old idea.⁵ It’s the backbone of search engines, recommender systems, log analytics, etc. Many retrieval algorithms developed for traditional retrieval systems can also be used for RAG. For instance, information retrieval is a fertile research area with a large supporting industry that can hardly be sufficiently covered within a few pages. Accordingly, this section will cover only the broad strokes. See this book’s [GitHub repository](#) for more in-depth resources on information retrieval.



Retrieval is typically limited to one database or system, whereas search involves retrieval across various systems. This chapter uses retrieval and search interchangeably.

At its core, retrieval works by ranking documents based on their relevance to a given query. Retrieval algorithms differ based on how relevance scores are computed. I’ll start with two common retrieval mechanisms: term-based retrieval and embedding-based retrieval.

⁵ Information retrieval was described as early as the 1920s in Emanuel Goldberg’s patents for a “statistical machine” to search documents stored on films. See “[The History of Information Retrieval Research](#)” (Sanderson and Croft, *Proceedings of the IEEE, 100: Special Centennial Issue*, April 2012).

Sparse Versus Dense Retrieval

In the literature, you might encounter the division of retrieval algorithms into the following categories: sparse versus dense. This book, however, opted for term-based versus embedding-based categorization.

Sparse retrievers represent data using *sparse vectors*. A sparse vector is a vector where the majority of the values are 0. Term-based retrieval is considered sparse, as each term can be represented using a sparse *one-hot vector*, a vector that is 0 everywhere except one value of 1. The vector size is the length of the vocabulary. The value of 1 is in the index corresponding to the index of the term in the vocabulary.

If we have a simple dictionary, {“food”: 0, “banana”: 1, “slug”: 2}, then the one-hot vectors of “food”, “banana”, and “slug” are [1, 0, 0], [0, 1, 0], and [0, 0, 1], respectively.

Dense retrievers represent data using *dense vectors*. A dense vector is a vector where the majority of the values aren’t 0. Embedding-based retrieval is typically considered dense, as embeddings are generally dense vectors. However, there are also sparse embeddings. For example, SPLADE (Sparse Lexical and Expansion) is a retrieval algorithm that works using sparse embeddings (Formal et al., 2021). It leverages embeddings generated by BERT but uses regularization to push most embedding values to 0. The sparsity makes embedding operations more efficient.

The sparse versus dense division causes SPLADE to be grouped together with term-based algorithms, even though SPLADE’s operations, strengths, and weaknesses are much more similar to those of dense embedding retrieval than those of term-based retrieval. Term-based versus embedding-based division avoids this miscategorization.

Term-based retrieval

Given a query, the most straightforward way to find relevant documents is with keywords. Some people call this approach *lexical retrieval*. For example, given the query “AI engineering”, the model will retrieve all the documents that contain “AI engineering”. However, this approach has two problems:

- Many documents might contain the given term, and your model might not have sufficient context space to include all of them as context. A heuristic is to include the documents that contain the term the greatest number of times. The assumption is that the more a term appears in a document, the more relevant this document is to this term. The number of times a term appears in a document is called *term frequency* (TF).
- A prompt can be long and contain many terms. Some are more important than others. For example, the prompt “Easy-to-follow recipes for Vietnamese food to cook at home” contains nine terms: *easy-to-follow*, *recipes*, *for*, *vietnamese*, *food*,

to, cook, at, home. You want to focus on more informative terms like *vietnamese* and *recipes*, not *for* and *at*. You need a way to identify important terms.

An intuition is that the more documents contain a term, the less informative this term is. “For” and “at” are likely to appear in most documents, hence, they are less informative. So a term’s importance is inversely proportional to the number of documents it appears in. This metric is called *inverse document frequency* (IDF). To compute IDF for a term, count all the documents that contain this term, then divide the total number of documents by this count. If there are 10 documents and 5 of them contain a given term, then the IDF of this term is $10 / 5 = 2$. The higher a term’s IDF, the more important it is.

TF-IDF is an algorithm that combines these two metrics: term frequency (TF) and inverse document frequency (IDF). Mathematically, the TF-IDF score of document D for the query Q is computed as follows:

- Let $t_1, t_2, \dots, t_q$ be the terms in the query Q .
- Given a term t , the term frequency of this term in the document D is $f(t, D)$.
- Let N be the total number of documents, and $C(t)$ be the number of documents that contain t . The IDF value of the term t can be written as $\text{IDF}(t) = \log \frac{N}{C(t)}$.
- Naively, the TF-IDF score of a document D with respect to Q is defined as $\text{Score}(D, Q) = \sum_{i=1}^q \text{IDF}(t_i) \times f(t_i, D)$.

Two common term-based retrieval solutions are Elasticsearch and BM25. **Elasticsearch** (Shay Banon, 2010), built on top of **Lucene**, uses a data structure called an inverted index. It’s a dictionary that maps from terms to documents that contain them. This dictionary allows for fast retrieval of documents given a term. The index might also store additional information such as the term frequency and the document count (how many documents contain this term), which are helpful for computing TF-IDF scores. **Table 6-1** illustrates an inverted index.

Table 6-1. A simplified example of an inverted index.

Term	Document count	(Document index, term frequency) for all documents containing the term
banana	2	(10, 3), (5, 2)
machine	4	(1, 5), (10, 1), (38, 9), (42, 5)
learning	3	(1, 5), (38, 7), (42, 5)
...	...	...

Okapi BM25, the 25th generation of the Best Matching algorithm, was developed by Robertson et al. in the 1980s. Its scorer is a modification of TF-IDF. Compared to naive TF-IDF, BM25 normalizes term frequency scores by document length. Longer

documents are more likely to contain a given term and have higher term frequency values.⁶

BM25 and its variances (BM25+, BM25F) are still widely used in the industry and serve as formidable baselines to compare against modern, more sophisticated retrieval algorithms, such as embedding-based retrieval, discussed next.⁷

One process I glossed over is tokenization, the process of breaking a query into individual terms. The simplest method is to split the query into words, treating each word as a separate term. However, this can lead to multi-word terms being broken into individual words, losing their original meaning. For example, “hot dog” would be split into “hot” and “dog”. When this happens, neither retains the meaning of the original term. One way to mitigate this issue is to treat the most common n-grams as terms. If the bigram “hot dog” is common, it’ll be treated as a term.

Additionally, you might want to convert all characters to lowercase, remove punctuation, and eliminate stop words (like “the”, “and”, “is”, etc.). Term-based retrieval solutions often handle these automatically. Classical NLP packages, such as **NLTK** (Natural Language Toolkit), **spaCy**, and **Stanford’s CoreNLP**, also offer tokenization functionalities.

Chapter 4 discusses measuring the lexical similarity between two texts based on their n-gram overlap. Can we retrieve documents based on the extent of their n-gram overlap with the query? Yes, we can. This approach works best when the query and the documents are of similar lengths. If the documents are much longer than the query, the likelihood of them containing the query’s n-grams increases, leading to many documents having similarly high overlap scores. This makes it difficult to distinguish truly relevant documents from less relevant ones.

Embedding-based retrieval

Term-based retrieval computes relevance at a lexical level rather than a semantic level. As mentioned in **Chapter 3**, the appearance of a text doesn’t necessarily capture its meaning. This can result in returning documents irrelevant to your intent. For example, querying “transformer architecture” might return documents about the electric device or the movie *Transformers*. On the other hand, *embedding-based retrievers* aim to rank documents based on how closely their meanings align with the query. This approach is also known as *semantic retrieval*.

6 For those interested in learning more about BM25, I recommend this paper by the BM25 authors: “**The Probabilistic Relevance Framework: BM25 and Beyond**” (Robertson and Zaragoza, *Foundations and Trends in Information Retrieval* 3 No. 4, 2009)

7 **Aravind Srinivas, the CEO of Perplexity**, tweeted that “Making a genuine improvement over BM25 or full-text search is hard”.

With embedding-based retrieval, indexing has an extra function: converting the original data chunks into embeddings. The database where the generated embeddings are stored is called a *vector database*. Querying then consists of two steps, as shown in [Figure 6-3](#):

1. **Embedding model**: convert the query into an embedding using the same embedding model used during indexing.
2. **Retriever**: fetch k data chunks whose embeddings are closest to the query embedding, as determined by the retriever. The number of data chunks to fetch, k , depends on the use case, the generative model, and the query.

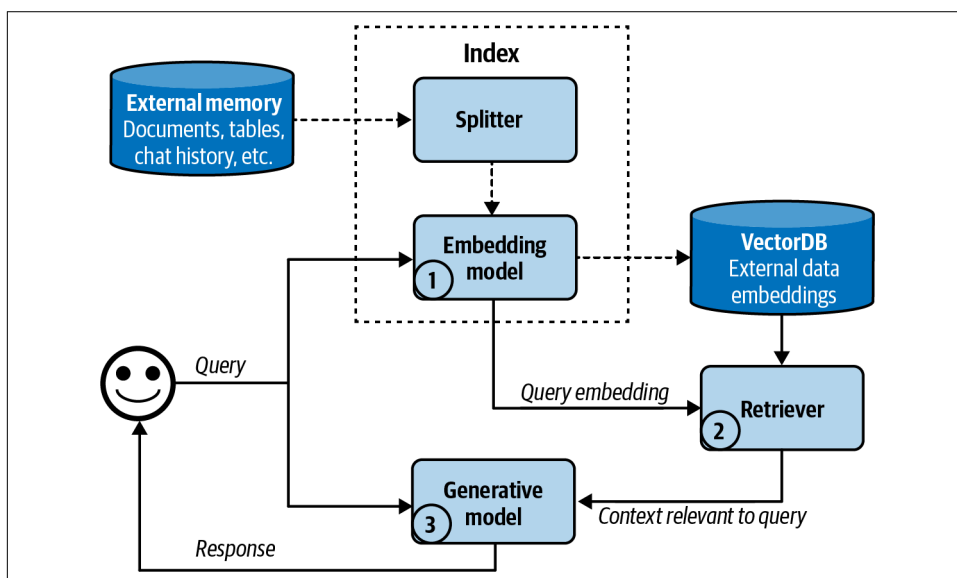


Figure 6-3. A high-level view of how an embedding-based, or semantic, retriever works.

The embedding-based retrieval workflow shown here is simplified. Real-world semantic retrieval systems might contain other components, such as a reranker to rerank all retrieved candidates, and caches to reduce latency.⁸

With embedding-based retrieval, we again encounter embeddings, which are discussed in [Chapter 3](#). As a reminder, an embedding is typically a vector that aims to preserve the important properties of the original data. An embedding-based retriever doesn't work if the embedding model is bad.

⁸ A RAG retrieval workflow shares many similar steps with the traditional recommender system.

Embedding-based retrieval also introduces a new component: vector databases. A vector database stores vectors. However, storing is the easy part of a vector database. The hard part is vector search. Given a query embedding, a vector database is responsible for finding vectors in the database close to the query and returning them. Vectors have to be indexed and stored in a way that makes vector search fast and efficient.

Like many other mechanisms that generative AI applications depend on, vector search isn't unique to generative AI. Vector search is common in any application that uses embeddings: search, recommendation, data organization, information retrieval, clustering, fraud detection, and more.

Vector search is typically framed as a nearest-neighbor search problem. For example, given a query, find the k nearest vectors. The naive solution is k -nearest neighbors (k -NN), which works as follows:

1. Compute the similarity scores between the query embedding and all vectors in the database, using metrics such as cosine similarity.
2. Rank all vectors by their similarity scores.
3. Return k vectors with the highest similarity scores.

This naive solution ensures that the results are precise, but it's computationally heavy and slow. It should be used only for small datasets.

For large datasets, vector search is typically done using an approximate nearest neighbor (ANN) algorithm. Due to the importance of vector search, many algorithms and libraries have been developed for it. Some popular vector search libraries are *FAISS* (Facebook AI Similarity Search) (Johnson et al., 2017), Google's *ScaNN* (Scalable Nearest Neighbors) (Sun et al., 2020), Spotify's *Annoy* (Bernhardsson, 2013), and *Hnswlib* (Hierarchical Navigable Small World) (Malkov and Yashunin, 2016).

Most application developers won't implement vector search themselves, so I'll give only a quick overview of different approaches. This overview might be helpful as you evaluate solutions.

In general, vector databases organize vectors into buckets, trees, or graphs. Vector search algorithms differ based on the heuristics they use to increase the likelihood that similar vectors are close to each other. Vectors can also be quantized (reduced precision) or made sparse. The idea is that quantized and sparse vectors are less computationally intensive to work with. For those wanting to learn more about vector search, Zilliz has an excellent [series](#) on it. Here are some significant vector search algorithms: